



Módulo 2 · Refactorización y Mejora de Código · Nivel Intermedio

Refactorización y Mejora de Código con IA

🔖 **Nivel Intermedio** · Requiere VS Code y herramienta de IA

Nombre del alumno:

ANGEL DAVID ECHENIQUE VILLANUEVA

Fecha:

03/03/2026

🔖 Instrucciones

Este ejercicio requiere VS Code y una herramienta de IA (Copilot, ChatGPT, Gemini...). No basta con que el código funcione: debes aplicar los tipos de refactorización del módulo de forma consciente.

Entrega: código refactorizado en VS Code + respuestas escritas justificando cada decisión.

BLOQUE
E 1

Análisis y simplificación en la práctica

La IA detecta code smells mucho mejor cuando le das contexto. En este bloque aplicarás el Paso 1 del workflow — Análisis inicial — sobre código real.

🔖 Ejercicio práctico — Abre VS Code para completar esta tarea

Ejercicio 1.1 Análisis inicial con IA

Copia este fragmento de código en VS Code y pide a la IA que lo analice buscando code smells:

```
def p(d, t, dc=0):
    r = 0
    for i in d:
        if t == 'A':
            if i['s'] == 1:
                r += i['p'] *
            i['q']
            elif t == 'B':
                if i['s'] == 1:
                    r += i['p'] * i['q'] * 0.9
                elif i['s'] == 2:
                    r += i['p'] * i['q'] * 0.8
            else:
                r += i['p'] *
            i['q']
            if dc > 0:
                r
    = r - (r * dc / 100)
    return
    r
```

 Prompt de análisis que usé:

Tengo un código smell y necesito ayuda para entenderlo y refactorizarlo aparte quiero que me digas los problemas que has detectado y donde.

Quiero saber los tipos de refactorización necesarios para este código y quiero usar funciones auxiliares.

Completa la tabla con los problemas que identificó la IA:

Code smell detectado	Dónde aparece	Tipo de refactorización necesario
Los nombres no son descriptivos	(p,d,t,dc,r,i,s,p,q)	Rename variables
Código duplicado	i['p'] * i['q']	Extract Variable / DRY
Condicionales anidadas, No son claras.	if t == 'A', elif t == 'B' con if internos	Simplificación de condicionales
Nos falta separar responsabilidades	La función calcula precios, aplica descuentos por tipo y descuento global	Modularización



Versión aún más limpia usando funciones auxiliares

Si quieres un diseño más profesional:

Python



Copiar

```
def obtener_factor(tipo, estado):
    reglas = {
        'A': {1: 1.0},
        'B': {1: 0.9, 2: 0.8},
    }
    return reglas.get(tipo, {}).get(estado, 1.0)

def calcular_precio(items, tipo, descuento_global=0):
    total = sum(
        item['p'] * item['q'] * obtener_factor(tipo, item['s'])
        for item in items
    )

    if descuento_global > 0:
        total *= (1 - descuento_global / 100)

    return total
```

Ejercicio 1.2 Aplicar Mejora de Nomenclatura

Antes de tocar la lógica, aplica solo refactorización de nomenclatura. Escribe el prompt que usarías y luego rellena la tabla con los nuevos nombres:

 **Prompt de mejora de nomenclatura:**

Necesito refactorizar la nomenclatura con descripciones claras. hazlo como si fueras mi jefe señor.

Nombre original	Nombre propuesto por la IA	¿Lo aceptas? ¿Por qué?
p()	Calcular_total_con_descuentos	Porque es descriptivo y es aceptable para producción.
d	Items	Porque asi sabremos que es una lista de precio, cantidad y estado.
t	Tipo_tarifa o categoria_precio	Porque es descriptivo y visible y entendible solo al leelo.
dc	Descuento_global	Porque dc no dice nada, pero descuento global es mucho mejor para mi y para mis compañeros en el futuro.
r	Total	Porque es descriptivo
i	Item	No, porque se parece a ITEMS. Lo cambiaria por OBJETO o PRENDAS.
s	Estado o nivel	Si porque es mas descriptivo que S...

Código refactorizado solo cambiando nomenclatura (sin alterar la lógica)

Python ^

 Copiar

```
def calcular_precio_total(items, tipo_tarifa, descuento_global=0):
    total = 0

    for item in items:
        precio = item['precio_unitario']
        cantidad = item['cantidad']
        estado = item['estado']

        if tipo_tarifa == 'A':
            if estado == 1:
                total += precio * cantidad

        elif tipo_tarifa == 'B':
            if estado == 1:
                total += precio * cantidad * 0.9
            elif estado == 2:
                total += precio * cantidad * 0.8

        else:
            total += precio * cantidad

    if descuento_global > 0:
        total -= total * descuento_global / 100

    return total
```

Ejercicio 1.3 Simplificación de condicionales

El código tiene condicionales anidados. Pide a la IA que los simplifique usando early returns o constantes semánticas. Escribe el prompt y analiza el resultado:

Prompt de simplificación:

En este código tengo condicionales anidados por lo tanto quiero que me lo simplifique usando early returns o constantes semánticas, como un señor.

¿Qué técnica usó la IA para simplificar?

- Eliminación de condicionales anidados y ha agregado bloques lineales.
- early-continue el cual nos ayuda que el flujo sea directo y predecible.
- Le dimos nombre descriptivos los numbers magic .

¿El código resultante es más legible?

Si porque le hemos dado nombre claros a nuestras variables y nuestro flujo es lineal, las reglas de negocios son explícitas y separadas por bloques.

Por otro lado centralizamos el cálculo subtotal y las conts semánticas explican el propósito de los multiplicadores.

Todo esto hace que nuestro código sea mantenible.

¿Cambió el comportamiento? Justifica:

No, porque solo he simplificado el código y añadido nomenclatura clara. La lógica de negocio sigue haciendo lo mismo, ya que solo he organizado el flujo.

- No altere el orden de acumulación ni la forma de recorrer los items.
- Cada condición reproduce exactamente las mismas reglas que el código original.
- El cálculo del descuento global es el mismo.

En pocas palabras lo que he hecho refactorizar el código puramente estructural, no funcional.



Código simplificado con early-continue y constantes semánticas

Python ^

 Copiar

```
DESCUENTO_B_ESTADO_1 = 0.90
DESCUENTO_B_ESTADO_2 = 0.80

def calcular_total(items, tipo_tarifa, descuento_global=0):
    total = 0

    for item in items:
        precio = item["p"]
        cantidad = item["q"]
        estado = item["s"]

        subtotal = precio * cantidad

        # Tarifa A
        if tipo_tarifa == "A":
            if estado == 1:
                total += subtotal
            continue

        # Tarifa B
        if tipo_tarifa == "B":
            if estado == 1:
                total += subtotal * DESCUENTO_B_ESTADO_1
            elif estado == 2:
                total += subtotal * DESCUENTO_B_ESTADO_2
            continue

        # Tarifa por defecto
        total += subtotal

    if descuento_global > 0:
        total -= total * (descuento_global / 100)

    return total
```



Modularización y patrones de diseño

Dividir funciones grandes y aplicar patrones son dos de los tipos de refactorización más potentes. Vamos a practicarlos sobre el mismo código del Bloque 1, ya renombrado.

Ejercicio práctico — Abre VS Code para completar esta tarea

Ejercicio 2.1 Modularizar la función

La función tiene varias responsabilidades mezcladas. Pide a la IA que la divida en funciones más pequeñas con responsabilidad única (principio SRP). Documenta el proceso:

Prompt de modularización:

Modularización y patrones de diseño

```
def p(d, t, dc=0): r = 0 for i in d: if t == 'A': if i['s'] == 1: r += i['p'] * i['q'] elif t == 'B': if i['s'] == 1: r += i['p'] * i['q'] * 0.9 elif i['s'] == 2: r += i['p'] * i['q'] * 0.8 else: r += i['p'] * i['q'] if dc > 0: r = r - (r * dc / 100) return r
```

necesito dividir funciones grandes y aplicar patrones de diseños a este código.

Esta función tiene varias responsabilidades mezclada por lo tanto quiero funciones mas pequeñas con una unica responsabilidad. Quiero que me documentes el proceso con docstrings como un jefe señor.

Función original (responsabilidad)	Función nueva (nombre sugerido)	¿Tiene responsabilidad única?
Calcular precio según tipo cliente	➤ tarifa_A(item) ➤ tarifa_B(item) ➤ tarifa_default(item)	Si, porque cada funcion aplica la regla de negocio de su tarifa indicada.
Aplicar descuento por stock	➤ tarifa_A, ➤ tarifa_B ➤ Yo cambiaria esto por nombre mas descriptivos.Ej: descuento por prendas viejas, descuento por compras comprar al mayor.	➤ Si, cada funcion decide, el importe segun el estado del items.
Aplicar descuento adicional	aplicar_descuento_global(total, descuento)	Si. Porque solo aplicamos el descuento final, sin tocar reglas de tarifa.

Orquestar el cálculo total	Calcular_total(items, tipo_tarifa, descuento_global)	➤ Si, solo coordinamos que el proceso sea mediante funciones especializadas.
----------------------------	--	--

Código modularizado con patrón Strategy y docstrings

Python ^

 Copiar

```
DESCUENTO_B_ESTADO_1 = 0.90
```

```
DESCUENTO_B_ESTADO_2 = 0.80
```

```
def calcular_subtotal(item):
```

```
    """
```

```
    Calcula el subtotal de un ítem individual.
```

```
    Responsabilidad:
```

- Multiplicar precio por cantidad.
- No aplica reglas de negocio ni descuentos.

```
    Parámetros:
```

```
    item (dict): Diccionario con claves 'p' (precio) y 'q' (cantidad).
```

```
    Retorna:
```

```
    float: Subtotal del ítem.
```

```
    """
```

```
    return item["p"] * item["q"]
```

```
def tarifa_A(item):
```

```
    """
```

```
    Estrategia de cálculo para la tarifa tipo A.
```

```
    Regla de negocio:
```

- Solo suma ítems cuyo estado sea 1.
- No aplica descuentos adicionales.

```
    Parámetros:
```

```
    item (dict): Ítem con clave 's' indicando el estado.
```

```
    Retorna:
```

```
    float: Importe a sumar para este ítem.
```

```
    """
```

```
    if item["s"] == 1:
```

```
        return calcular_subtotal(item)
```

```
    return 0
```

```
def tarifa_B(item):
    """
    Estrategia de cálculo para la tarifa tipo B.

    Regla de negocio:
    - Estado 1 → descuento del 10%.
    - Estado 2 → descuento del 20%.
    - Otros estados → no suma.

    Parámetros:
    item (dict): Ítem con clave 's' indicando el estado.

    Retorna:
    float: Importe a sumar para este ítem.
    """
    subtotal = calcular_subtotal(item)

    if item["s"] == 1:
        return subtotal * DESCUENTO_B_ESTADO_1
    if item["s"] == 2:
        return subtotal * DESCUENTO_B_ESTADO_2
    return 0

def tarifa_default(item):
    """
    Estrategia por defecto cuando la tarifa no coincide con A ni B.

    Regla de negocio:
    - Suma el subtotal sin descuentos ni condiciones.

    Parámetros:
    item (dict): Ítem a procesar.

    Retorna:
    float: Subtotal del ítem.
    """
    return calcular_subtotal(item)

def obtener_estrategia(tipo_tarifa):
    """
    Selecciona la estrategia de cálculo según el tipo de tarifa.

    Patrón aplicado:
    - Strategy Pattern: cada tarifa es una función intercambiable.

    Parámetros:
    tipo_tarifa (str): Identificador de tarifa ('A', 'B', etc.).

    Retorna:
```



```

Retorna:
    function: Función estrategia correspondiente.
"""
estrategias = {
    "A": tarifa_A,
    "B": tarifa_B
}
return estrategias.get(tipo_tarifa, tarifa_default)

def aplicar_descuento_global(total, descuento):
    """
    Aplica un descuento porcentual al total.

    Responsabilidad:
        - No calcula subtotales ni reglas de tarifa.
        - Solo aplica el descuento final.

    Parámetros:
        total (float): Importe acumulado.
        descuento (float): Porcentaje de descuento global.

    Retorna:
        float: Total tras aplicar el descuento.
    """
    if descuento > 0:
        return total * (1 - descuento / 100)
    return total

def calcular_total(items, tipo_tarifa, descuento_global=0):
    """
    Función orquestadora que calcula el total final.

    Responsabilidades:
        - Seleccionar la estrategia adecuada.
        - Iterar sobre los items.
        - Acumular subtotales según la estrategia.
        - Aplicar descuento global.

    Parámetros:
        items (list): Lista de items con precio, cantidad y estado.
        tipo_tarifa (str): Tipo de tarifa ('A', 'B', etc.).
        descuento_global (float): Descuento final en porcentaje.

    Retorna:
        float: Total final calculado.
    """
    estrategia = obtener_estrategia(tipo_tarifa)
    total = sum(estrategia(item) for item in items)
    return aplicar_descuento_global(total, descuento_global)

```

Ejercicio 2.2 Aplica el patrón Strategy

El código usa condicionales para decidir el comportamiento según el tipo de cliente. Este es el caso de uso perfecto para el patrón Strategy. Pide a la IA que lo refactorice usando este patrón:

Recuerda

El patrón Strategy encapsula cada algoritmo (tipo_cliente A, B, otro) en una clase separada. Así puedes añadir nuevos tipos sin modificar la función principal (principio Open/Closed).

Prompt de aplicación del patrón Strategy:

Al código anterior quiero refactorizarlo aplicando el patrón strategy y comentarlo con docstrings.

Código refactorizado con Strategy Pattern y docstrings

Python ^

 Copiar

```
DESCUENTO_B_ESTADO_1 = 0.90
DESCUENTO_B_ESTADO_2 = 0.80

def calcular_subtotal(item):
    """
    Calcula el subtotal de un ítem individual.

    Responsabilidad única:
    - Multiplicar precio por cantidad.
    """
    return item["p"] * item["q"]

class EstrategiaTarifa:
    """
    Interfaz base para todas las estrategias de tarifa.

    Patrón aplicado:
    - Strategy Pattern: cada tarifa implementa su propia lógica
      sin condicionales en la función principal.
    """

    def calcular(self, item):
        raise NotImplementedError("Debe implementarse en la subclase.")

class TarifaA(EstrategiaTarifa):
    """
    Estrategia de cálculo para tarifa tipo A.

    Regla de negocio:
    - Solo suma ítems cuyo estado sea 1.
    """

    def calcular(self, item):
        if item["s"] == 1:
            return calcular_subtotal(item)
        return 0

class TarifaB(EstrategiaTarifa):
    """
    Estrategia de cálculo para tarifa tipo B.

    Regla de negocio:
```

```

Regla de negocio:
- Estado 1 → descuento del 10%.
- Estado 2 → descuento del 20%.
"""

def calcular(self, item):
    subtotal = calcular_subtotal(item)

    if item["s"] == 1:
        return subtotal * DESCUENTO_B_ESTADO_1
    if item["s"] == 2:
        return subtotal * DESCUENTO_B_ESTADO_2
    return 0

class TarifaDefault(EstrategiaTarifa):
    """
    Estrategia por defecto cuando la tarifa no coincide con A ni B.

    Regla de negocio:
    - Suma el subtotal sin descuentos.
    """

    def calcular(self, item):
        return calcular_subtotal(item)

def obtener_estrategia(tipo_tarifa):
    """
    Devuelve la estrategia adecuada según el tipo de tarifa.

    Responsabilidad única:
    - Seleccionar la estrategia correcta sin condicionales en la función principal.
    """
    estrategias = {
        "A": TarifaA(),
        "B": TarifaB()
    }
    return estrategias.get(tipo_tarifa, TarifaDefault())

def aplicar_descuento_global(total, descuento):
    """
    Aplica un descuento porcentual al total.

    Responsabilidad única:
    - Aplicar el descuento final sin modificar reglas de tarifa.
    """
    if descuento > 0:
        return total * (1 - descuento / 100)
    return total

```

```

def calcular_total(items, tipo_tarifa, descuento_global=0):
    """
    Función orquestadora del cálculo total.

    Responsabilidades:
    - Seleccionar la estrategia.
    - Iterar sobre los items.
    - Acumular subtotales según la estrategia.
    - Aplicar descuento global.

    Esta función ya no contiene reglas de negocio.
    """
    estrategia = obtener_estrategia(tipo_tarifa)
    total = sum(estrategia.calcular(item) for item in items)
    return aplicar_descuento_global(total, descuento_global)

```

¿Qué ventaja concreta ofrece la versión con Strategy frente a los if/elif originales?

Eliminación total de condicionales anidadas, ahora cada tarifa vive en su propia clase por lo tanto cada una tiene su propia responsabilidad.

¿Qué pasa si mañana hay que añadir un tipo de cliente 'C'?

Con lo if/elif para agregar cliente c tendríamos que abrir la función añadir un nuevo bloque elif y repetir lógica, podríamos romper algo que ya funciona y esto violaría el open/closed principle (ocp). Pero con strategy es tan simple como agregar una clase de tarifa c y registrarla y ya. No tocamos ninguna función principal, ni ninguna otra estrategia aparte podemos hacer testeos mucho más fáciles.

Ejercicio 2.3 Comparativa antes/después

Completa esta tabla comparando el código original con el refactorizado al finalizar los dos ejercicios anteriores:

Métrica	Código original	Código refactorizado
Nº de funciones/métodos	1 hacia todo una sola función	Varias funciones pequeñas más 3 clases de estrategia.
Líneas de la función principal	~18 muchas líneas para una sola función	Calcular total resumida en 3 líneas.
¿Tiene nombres descriptivos?	No, p,d,t,r,dc.	Nomenclatura clara y descriptiva
¿Fácil de añadir tipo cliente nuevo?	No (hay que modificar if/elif)	Ahora podemos crear una nueva clase de estrategia sin tocar nada de código que ya funciona.
¿Se puede testear cada parte por separado?	Difícil, si porque se testea todo de golpe.	Podemos testear todo de forma independiente.



BLOQUE
E 3

Workflow profesional — Proyecto completo

En este bloque aplicarás el workflow de 5 pasos completo sobre un fragmento de código legacy real.

 Ejercicio práctico — Abre VS Code para completar esta tarea

Código a refactorizar — sistema de gestión de usuarios legacy:

```
def process(u, pw, r, db):
    import hashlib
    h = hashlib.md5(pw.encode()).hexdigest()
    for x in db:
        if x[0] == u and x[1] == h:
            if r == 'admin':
                return {'ok': True, 'lvl': 3, 'msg': 'Admin OK'}
            elif r == 'user':
                return {'ok': True, 'lvl': 1, 'msg': 'User OK'}
            else:
                return {'ok': True, 'lvl': 0, 'msg': 'Guest OK'}
    return {'ok': False, 'lvl': -1, 'msg': 'Auth failed'}
```

Ejercicio 3.1 Aplica los 5 pasos del workflow

Para cada paso, escribe el prompt exacto que usaste y qué detectó o generó la IA:

Paso 1
Análisis
inicial

Identifica code smells, problemas de seguridad y áreas de mejora

Prompt exacto:

Quiero que me analices este fragmento de código legacy real y me le aplique los 5 pasos de workflow. quiero también que me explique que hiciste.

¿Qué detectó/generó la IA?

- **Mezcla de responsabilidades:** hashing, autenticación, selección de rol, construcción de respuesta.
- **Nombres poco descriptivos:** u, pw, r, db, x.
- **Uso inseguro de MD5.**
- **Condicionales rígidos** para roles.
- **Difícil de testear** porque todo está acoplado.
- **No hay separación de capas** (servicio, repositorio, lógica de negocio).
- **Retornos múltiples** que dificultan trazabilidad.

¿Qué ajustaste tú?

1. Revise el cambio de nomenclatura y ajuste acorde a como yo quería
2. Repartí responsabilidades creando archivos y carpetas.

➤ Servicio principal

Servicio principal de autenticación

Python

 Copiar

```
class AuthService:
    """
    Servicio encargado de procesar la autenticación completa.

    Responsabilidades:
    - Validar entradas.
    - Hashear contraseña.
    - Consultar repositorio.
    - Seleccionar estrategia de rol.
    - Construir respuesta final.
    """

    def __init__(self, repo: UserRepository, hasher: Hasher):
        self.repo = repo
        self.hasher = hasher

    def process(self, username: str, password: str, role: str) -> dict:
        """
        Procesa la autenticación de un usuario.

        Parámetros:
            username (str): Nombre de usuario.
            password (str): Contraseña en texto plano.
            role (str): Rol solicitado.

        Retorna:
            dict: Resultado de autenticación.

        Excepciones:
            ValueError: Si username o password son inválidos.
            Exception: Para errores inesperados.
        """
        if not isinstance(username, str) or not username.strip():
            raise ValueError("El nombre de usuario no puede estar vacío.")

        try:
            hashed = self.hasher.hash(password)
        except Exception as e:
            raise Exception(f"Error al hashear la contraseña: {e}")

        try:
            authenticated = self.repo.authenticate(username, hashed)
        except Exception as e:
            raise Exception(f"Error al acceder al repositorio: {e}")

        if not authenticated:
            return {'ok': False, 'lvl': -1, 'msg': 'Auth failed'}

        strategy = get_role_strategy(role)
        return strategy.build_response()
```

➤ Hasher

Hasher

Python ^ Copiar

```
import hashlib

class Hasher:
    """
    Servicio responsable de generar hashes de contraseñas.

    Notas:
    - En un entorno real se debería usar bcrypt o argon2.
    - Esta clase encapsula el hashing para permitir testeo y reemplazo futuro.
    """

    def hash(self, password: str) -> str:
        """
        Devuelve el hash MD5 de una contraseña.

        Parámetros:
            password (str): Contraseña en texto plano.

        Retorna:
            str: Hash MD5 de la contraseña.

        Excepciones:
            ValueError: Si la contraseña está vacía o no es str.
        """
        if not isinstance(password, str) or not password.strip():
            raise ValueError("La contraseña no puede estar vacía.")

        return hashlib.md5(password.encode()).hexdigest()
```

➤ Repositorio de usuarios

Repositorio de usuarios

Python ^ Copiar

```
class UserRepository:
    """
    Repositorio encargado de acceder a la base de datos de usuarios.

    La base de datos se espera como una lista de tuplas:
    [
        ("usuario", "hash_password"),
        ...
    ]
    """

    def __init__(self, database: list):
        if not isinstance(database, list):
            raise TypeError("La base de datos debe ser una lista.")
        self.database = database

    def authenticate(self, username: str, hashed_password: str) -> bool:
        """
        Verifica si existe un usuario con el hash de contraseña dado.

        Parámetros:
            username (str): Nombre de usuario.
            hashed_password (str): Hash MD5 de la contraseña.

        Retorna:
            bool: True si coincide usuario y contraseña, False en caso contrario.
        """
        for user, pw in self.database:
            if user == username and pw == hashed_password:
                return True
        return False
```

➤ Estrategia de rol

Estrategias de rol (Strategy Pattern)

Python ^

 Copiar

```
class RoleStrategy:
    """
    Interfaz base para todas las estrategias de rol.
    Cada rol define su nivel y mensaje.
    """

    def build_response(self) -> dict:
        raise NotImplementedError("Debe implementarse en la subclase.")
```

Python ^

 Copiar

```
class AdminRole(RoleStrategy):
    """Estrategia para rol administrador."""

    def build_response(self) -> dict:
        return {'ok': True, 'lvl': 3, 'msg': 'Admin OK'}
```

Python ^

 Copiar

```
class UserRole(RoleStrategy):
    """Estrategia para rol usuario estándar."""

    def build_response(self) -> dict:
        return {'ok': True, 'lvl': 1, 'msg': 'User OK'}
```

Python ^

 Copiar

```
class GuestRole(RoleStrategy):
    """Estrategia para rol invitado."""

    def build_response(self) -> dict:
        return {'ok': True, 'lvl': 0, 'msg': 'Guest OK'}
```

➤ Selector de estrategia

Selector de estrategia

Python ^

 Copiar

```
def get_role_strategy(role: str) -> RoleStrategy:
    """
    Devuelve la estrategia correspondiente al rol solicitado.

    Parámetros:
        role (str): Nombre del rol.

    Retorna:
        RoleStrategy: Estrategia asociada al rol.
    """
    roles = {
        'admin': AdminRole(),
        'user': UserRole()
    }
    return roles.get(role, GuestRole())
```

➤ Añadi Tests

Estructura del proyecto

```
Código ^ Copiar

project/
├── auth/
│   ├── hasher.py
│   ├── repository.py
│   ├── roles.py
│   └── service.py
└── tests/
    ├── test_hasher.py
    ├── test_repository.py
    ├── test_roles.py
    └── test_auth_service.py
```

```
test_hasher.py

Python ^ Copiar

import pytest
from auth.hasher import Hasher

def test_hash_generates_md5_correctly():
    hasher = Hasher()
    result = hasher.hash("password")
    assert result == "5f4dcc3b5aa765d61d8327deb882cf99" # MD5 conocido

def test_hash_raises_error_on_empty_password():
    hasher = Hasher()
    with pytest.raises(ValueError):
        hasher.hash("")

def test_hash_raises_error_on_non_string():
    hasher = Hasher()
    with pytest.raises(ValueError):
        hasher.hash(123)
```

```
test_repository.py

Python ^ Copiar

from auth.repository import UserRepository

def test_authenticate_success():
    db = [("angel", "abc123")]
    repo = UserRepository(db)
    assert repo.authenticate("angel", "abc123") is True

def test_authenticate_failure_wrong_password():
    db = [("angel", "abc123")]
    repo = UserRepository(db)
    assert repo.authenticate("angel", "wrong") is False

def test_authenticate_failure_user_not_found():
    db = [("angel", "abc123")]
    repo = UserRepository(db)
    assert repo.authenticate("otro", "abc123") is False

def test_repository_raises_if_not_list():
    try:
        UserRepository("not a list")
        assert False, "Debe lanzar TypeError"
    except TypeError:
        assert True
```

test_roles.py

Python ^

 Copiar

```
from auth.roles import AdminRole, UserRole, GuestRole

def test_admin_role_response():
    r = AdminRole().build_response()
    assert r["ok"] is True
    assert r["lvl"] == 3
    assert r["msg"] == "Admin OK"

def test_user_role_response():
    r = UserRole().build_response()
    assert r["lvl"] == 1

def test_guest_role_response():
    r = GuestRole().build_response()
    assert r["lvl"] == 0
```

test_auth_service.py

Python ^

 Copiar

```
import pytest
from auth.service import AuthService
from auth.repository import UserRepository
from auth.hasher import Hasher
from auth.roles import AdminRole, UserRole, GuestRole

class FakeHasher:
    """Hasher controlado para tests."""
    def hash(self, pw):
        return "hashed"

class FakeRepo:
    """Repositorio controlado para tests."""
    def __init__(self, should_auth):
        self.should_auth = should_auth

    def authenticate(self, u, h):
        return self.should_auth

def test_auth_success_admin():
    repo = FakeRepo(True)
    hasher = FakeHasher()
    service = AuthService(repo, hasher)

    result = service.process("angel", "pw", "admin")
    assert result["lvl"] == 3
    assert result["msg"] == "Admin OK"

def test_auth_success_user():
    repo = FakeRepo(True)
    hasher = FakeHasher()
    service = AuthService(repo, hasher)

    result = service.process("angel", "pw", "user")
    assert result["lvl"] == 1

def test_auth_success_guest_default():
    repo = FakeRepo(True)
    hasher = FakeHasher()
    service = AuthService(repo, hasher)

    result = service.process("angel", "pw", "otro")
    assert result["lvl"] == 0

def test_auth_failure():
    repo = FakeRepo(False)
    hasher = FakeHasher()
    service = AuthService(repo, hasher)

    result = service.process("angel", "pw", "admin")
    assert result["ok"] is False
    assert result["lvl"] == -1
```

```
def test_auth_raises_on_empty_username():
    repo = FakeRepo(True)
    hasher = FakeHasher()
    service = AuthService(repo, hasher)

    with pytest.raises(ValueError):
        service.process("", "pw", "admin")

def test_auth_raises_on_hasher_error():
    class BadHasher:
        def hash(self, pw):
            raise Exception("hash error")

    repo = FakeRepo(True)
    service = AuthService(repo, BadHasher())

    with pytest.raises(Exception):
        service.process("angel", "pw", "admin")

def test_auth_raises_on_repo_error():
    class BadRepo:
        def authenticate(self, u, h):
            raise Exception("db error")

    hasher = FakeHasher()
    service = AuthService(BadRepo(), hasher)

    with pytest.raises(Exception):
        service.process("angel", "pw", "admin")
```

- Documentacion con docstrings
 - 3. Añadi manejo de excepciones y validaciones de usuario.
-

Paso 2	Propón un plan priorizado por impacto (seguridad primero)
Plan de refactorización	

🔍 Prompt exacto:

Quiero que me de un plan priorizado hacia la seguridad para implementar en este codigo. comentame lo que encuentres vulnerable.

¿Qué detectó/generó la IA?

- MD5 fuera, mejor Bcrypt o argon2.
- Necesito mas validaciones porque las que tengo falta de seguridad.
- Deberia proteger el repositorio del usuario faltan afinar cosas.
- no hay expiracion de session ni tokens.

¿Qué ajustaste tú?

- Cambien a BCrypt.
- Limite intentos de autentificacion
- logging seguro.

```
Python ^ Copiar

import bcrypt

class Hasher:
    def hash(self, password: str) -> str:
        if not password.strip():
            raise ValueError("La contraseña no puede estar vacía.")
        return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

    def verify(self, password: str, hashed: str) -> bool:
        return bcrypt.checkpw(password.encode(), hashed.encode())
```

Ajuste 2: Validar roles explícitamente

Python ^

 Copiar

```
VALID_ROLES = {"admin", "user", "guest"}

if role not in VALID_ROLES:
    role = "guest"
```

Ajuste 3: Limitar intentos de autenticación

Python ^

 Copiar

```
class AuthLimiter:
    attempts = {}

    def register_attempt(self, username):
        self.attempts[username] = self.attempts.get(username, 0) + 1
        if self.attempts[username] > 5:
            raise Exception("Cuenta bloqueada temporalmente.")
```

Ajuste 5: Logging seguro

Python ^

 Copiar

```
import logging
logger = logging.getLogger("auth")

logger.warning(f"Intento fallido de login para usuario: {username}")
```

Paso 3
Ejecu
ción
incre
ment
al

Ejecuta el primer cambio del plan (solo uno, no todo a la vez)

Prompt exacto:

Aplicame este cambio-> el hashing MD5 por bcrypt. Mantén toda la arquitectura, clases y métodos existentes sin modificar su estructura.

Asegúrate de que el nuevo hasher sea compatible con el repositorio y el AuthService. Y no hagas nada mas

¿Qué detectó/generó la IA?

1. Con MD5, tu sistema era vulnerable a fuerza bruta y tablas rainbow.
2. La comparación `hash == hash` ya no sirve con bcrypt, porque cada hash lleva un salt distinto.
3. Para usar bcrypt bien, necesitas:

- Un método verify.
- Que el repositorio conozca el hasher o al menos use verify en algún punto.

¿Qué ajustaste tú?

- Me asegure que el nuevo hasher tenga metodos hash() y verify().
- añadi un nuevo test para bcrypt.
- Todo quedo igual en mensaje roles expcepciones y validaciones.
- Añadí verify para comparar contraseña en texto plano con hash almacenado
- Ahora recibe password en texto plano y usa self.hasher.verify(...).

Hasher con bcrypt

```
Python ^ Copiar

import bcrypt

class Hasher:
    """
    Servicio responsable de generar y verificar hashes de contraseñas.

    Notas:
    - Usa bcrypt, adecuado para producción (lento y con salt).
    """

    def hash(self, password: str) -> str:
        """
        Genera un hash seguro con bcrypt.

        Lanza ValueError si la contraseña es inválida.
        """
        if not isinstance(password, str) or not password.strip():
            raise ValueError("La contraseña no puede estar vacía.")

        hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
        return hashed.decode()

    def verify(self, password: str, hashed_password: str) -> bool:
        """
        Verifica si una contraseña en texto plano coincide con el hash almacenado.
        """
        if not isinstance(password, str) or not isinstance(hashed_password, str):
            return False

        return bcrypt.checkpw(password.encode(), hashed_password.encode())
```

UserRepository usando verify en lugar de comparar strings

```
Python ^ Copiar

class UserRepository:
    """
    Repositorio encargado de acceder a la base de datos de usuarios.

    Ahora depende de Hasher para verificar contraseñas con bcrypt.
    """

    def __init__(self, database: list, hasher: Hasher):
        if not isinstance(database, list):
            raise TypeError("La base de datos debe ser una lista.")
        self.database = database
        self.hasher = hasher

    def authenticate(self, username: str, password: str) -> bool:
        """
        Verifica si existe un usuario con la contraseña dada.

        En la base de datos se espera almacenar el hash bcrypt.
        """
        for user, stored_hash in self.database:
            if user == username and self.hasher.verify(password, stored_hash):
                return True
        return False
```

AuthService ajustado mínimamente

```
Python ^ Copiar

class AuthService:
    """
    Servicio encargado de procesar la autenticación completa.
    """

    def __init__(self, repo: UserRepository, hasher: Hasher):
        # Mantenemos la firma original por compatibilidad,
        # aunque ahora el hasher se usa dentro del repositorio.
        self.repo = repo
        self.hasher = hasher

    def process(self, username: str, password: str, role: str) -> dict:
        """
        Procesa la autenticación de un usuario.
        """
        if not isinstance(username, str) or not username.strip():
            raise ValueError("El nombre de usuario no puede estar vacío.")

        try:
            authenticated = self.repo.authenticate(username, password)
        except Exception as e:
            raise Exception(f"Error al acceder al repositorio: {e}")

        if not authenticated:
            return {'ok': False, 'lvl': -1, 'msg': 'Auth failed'}

        strategy = get_role_strategy(role)
        return strategy.build_response()
```

Paso 4 Validación

Genera tests para verificar que el comportamiento no cambió

🔍 Prompt exacto:

necesito verificar que todo el comportamiento no haya cambiado. así que genere los tests necesarios para verificar que todo .

¿Qué detectó/generó la IA? ¿Qué ajustaste tú?

- El flujo original dependía de comparar hashes deterministas
- AuthService seguía hasheando la contraseña antes de enviarla al repositorio
- La base de datos de pruebas debe almacenar hashes bcrypt
- El repositorio ahora necesita el hasher inyectado

Tests del Hasher (bcrypt)

```
Python ^ Copiar

import pytest
from auth.hasher import Hasher
import bcrypt

def test_hash_generates_valid_bcrypt_hash():
    hasher = Hasher()
    hashed = hasher.hash("password")
    assert isinstance(hashed, str)
    assert hashed.startswith("$2b$") or hashed.startswith("$2a$")

def test_verify_correct_password():
    hasher = Hasher()
    hashed = hasher.hash("password")
    assert hasher.verify("password", hashed) is True

def test_verify_wrong_password():
    hasher = Hasher()
    hashed = hasher.hash("password")
    assert hasher.verify("wrong", hashed) is False

def test_hash_raises_on_empty_password():
    hasher = Hasher()
    with pytest.raises(ValueError):
        hasher.hash("")
```

Tests del UserRepository con bcrypt

```
Python ^ Copiar

from auth.repository import UserRepository
from auth.hasher import Hasher

def test_authenticate_success_with_bcrypt():
    hasher = Hasher()
    hashed = hasher.hash("secret")
    db = [("angel", hashed)]
    repo = UserRepository(db, hasher)

    assert repo.authenticate("angel", "secret") is True

def test_authenticate_failure_wrong_password():
    hasher = Hasher()
    hashed = hasher.hash("secret")
    db = [("angel", hashed)]
    repo = UserRepository(db, hasher)

    assert repo.authenticate("angel", "wrong") is False

def test_authenticate_failure_user_not_found():
    hasher = Hasher()
    hashed = hasher.hash("secret")
    db = [("angel", hashed)]
    repo = UserRepository(db, hasher)

    assert repo.authenticate("otro", "secret") is False
```

Tests de roles

```
Python ^ Copiar

from auth.roles import AdminRole, UserRole, GuestRole

def test_admin_role():
    r = AdminRole().build_response()
    assert r["lvl"] == 3

def test_user_role():
    r = UserRole().build_response()
    assert r["lvl"] == 1

def test_guest_role():
    r = GuestRole().build_response()
    assert r["lvl"] == 0
```

Tests del AuthService

```
Python ^ Copiar

import pytest
from auth.service import AuthService
from auth.repository import UserRepository
from auth.hasher import Hasher

def setup_auth():
    hasher = Hasher()
    hashed = hasher.hash("pw")
    db = [("angel", hashed)]
    repo = UserRepository(db, hasher)
    return AuthService(repo, hasher)

def test_auth_success_admin():
    service = setup_auth()
    result = service.process("angel", "pw", "admin")
    assert result["lvl"] == 3

def test_auth_success_user():
    service = setup_auth()
    result = service.process("angel", "pw", "user")
    assert result["lvl"] == 1

def test_auth_success_guest_default():
    service = setup_auth()
    result = service.process("angel", "pw", "otro")
    assert result["lvl"] == 0

def test_auth_failure():
    service = setup_auth()
    result = service.process("angel", "wrong", "admin")
    assert result["ok"] is False
    assert result["lvl"] == -1

def test_auth_raises_on_empty_username():
    service = setup_auth()
    with pytest.raises(ValueError):
        service.process("", "pw", "admin")
```

Yo podria mejora esto añadiendo

- Añadir tests de rendimiento (bcrypt es más lento).
 - Añadir tests de seguridad (contraseñas vacías, inputs raros).
 - Añadir tests de migración (MD5 → bcrypt).
 - Añadir tests de roles personalizados si amplías el sistema.
-

🔗 Prompt exacto: quiero actualizar mis docstring de todo lo que hemos agregado desde el principio hasta aqui por partes. como un señor.

¿Qué detectó/generó la IA? ¿Qué ajustaste tú?

hasher.py

Python ^

Copiar

```
import bcrypt

class Hasher:
    """
    Servicio responsable de generar y verificar hashes de contraseñas.

    Este componente encapsula toda la lógica criptográfica del sistema.
    Utiliza bcrypt, un algoritmo seguro diseñado para proteger contraseñas
    mediante sal aleatoria y un proceso de hashing deliberadamente lento.

    Responsabilidades:
    - Generar hashes seguros para almacenamiento.
    - Verificar contraseñas en texto plano contra hashes almacenados.
    - Validar entradas para evitar errores silenciosos o estados inválidos.

    Notas:
    - bcrypt genera un hash distinto cada vez debido al salt.
    - Nunca debe compararse un hash bcrypt mediante igualdad directa.
    """

    def hash(self, password: str) -> str:
        """
        Genera un hash seguro usando bcrypt.

        Args:
            password (str): Contraseña en texto plano.

        Returns:
            str: Hash bcrypt codificado en UTF-8.

        Raises:
            ValueError: Si la contraseña está vacía o no es una cadena válida.
        """
        if not isinstance(password, str) or not password.strip():
            raise ValueError("La contraseña no puede estar vacía.")

        hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
        return hashed.decode()

    def verify(self, password: str, hashed_password: str) -> bool:
        """
        Verifica si una contraseña coincide con un hash bcrypt almacenado.

        Args:
            password (str): Contraseña en texto plano.
            hashed_password (str): Hash bcrypt previamente almacenado.

        Returns:
            bool: True si la contraseña es válida, False en caso contrario.
        """
        if not isinstance(password, str) or not isinstance(hashed_password, str):
            return False

        return bcrypt.checkpw(password.encode(), hashed_password.encode())
```

repository.py

Python ^

 Copiar

```
class UserRepository:
    """
    Repositorio encargado de gestionar el acceso a la base de datos de usuarios.

    La base de datos se modela como una lista de tuplas:
    [
        ("usuario", "hash_bcrypt"),
        ...
    ]

    Este componente abstrae la capa de persistencia y centraliza la lógica
    de autenticación basada en bcrypt.

    Responsabilidades:
    - Consultar usuarios.
    - Verificar credenciales usando el hasher.
    - Mantener la base de datos en un formato consistente.
    """

    def __init__(self, database: list, hasher):
        """
        Inicializa el repositorio con una base de datos y un servicio de hashing.

        Args:
            database (list): Lista de tuplas (usuario, hash).
            hasher (Hasher): Servicio de hashing para verificar contraseñas.

        Raises:
            TypeError: Si la base de datos no es una lista.
        """
        if not isinstance(database, list):
            raise TypeError("La base de datos debe ser una lista.")

        self.database = database
        self.hasher = hasher

    def authenticate(self, username: str, password: str) -> bool:
        """
        Verifica si existe un usuario con la contraseña proporcionada.

        Args:
            username (str): Nombre de usuario.
            password (str): Contraseña en texto plano.

        Returns:
            bool: True si las credenciales son correctas, False en caso contrario.
        """
        for user, stored_hash in self.database:
            if user == username and self.hasher.verify(password, stored_hash):
                return True
        return False
```

selector.py

Python ^

 Copiar

```
def get_role_strategy(role: str):
    """
    Devuelve la estrategia correspondiente al rol solicitado.

    Args:
        role (str): Nombre del rol solicitado.

    Returns:
        RoleStrategy: Estrategia asociada al rol. GuestRole si el rol no existe.
    """
    roles = {
        'admin': AdminRole(),
        'user': UserRole()
    }
    return roles.get(role, GuestRole())
```

Python ^

 Copiar

```
class RoleStrategy:
    """
    Interfaz base para todas las estrategias de rol.

    Cada rol define su propio nivel de acceso y mensaje asociado.
    Este patrón permite extender el sistema sin modificar código existente.
    """

    def build_response(self) -> dict:
        """
        Construye la respuesta asociada al rol.

        Returns:
            dict: Diccionario con información de autenticación.

        Raises:
            NotImplementedError: Si la subclase no implementa el método.
        """
        raise NotImplementedError("Debe implementarse en la subclase.")
```

Python ^

 Copiar

```
class AdminRole(RoleStrategy):
    """
    Estrategia para el rol administrador.

    Nivel de acceso más alto del sistema.
    """

    def build_response(self) -> dict:
        return {'ok': True, 'lvl': 3, 'msg': 'Admin OK'}
```

Python ^

 Copiar

```
class UserRole(RoleStrategy):
    """
    Estrategia para el rol usuario estándar.
    """

    def build_response(self) -> dict:
        return {'ok': True, 'lvl': 1, 'msg': 'User OK'}
```

Python ^

 Copiar

```
class GuestRole(RoleStrategy):
    """
    Estrategia para el rol invitado.

    Rol por defecto cuando el rol solicitado no coincide con ninguno conocido.
    """

    def build_response(self) -> dict:
        return {'ok': True, 'lvl': 0, 'msg': 'Guest OK'}
```

```
class AuthService:
    """
    Servicio principal encargado de procesar la autenticación del usuario.

    Este componente orquesta el flujo completo:
    - Validación de entradas.
    - Delegación de autenticación al repositorio.
    - Selección de estrategia de rol.
    - Construcción de la respuesta final.

    El servicio no contiene reglas de negocio específicas, lo que facilita
    su testeo y mantenimiento.
    """

    def __init__(self, repo, hasher):
        """
        Inicializa el servicio con un repositorio y un hasher.

        Args:
            repo (UserRepository): Repositorio de usuarios.
            hasher (Hasher): Servicio de hashing (mantenido por compatibilidad).
        """
        self.repo = repo
        self.hasher = hasher

    def process(self, username: str, password: str, role: str) -> dict:
        """
        Procesa la autenticación de un usuario y devuelve la respuesta final.

        Args:
            username (str): Nombre de usuario.
            password (str): Contraseña en texto plano.
            role (str): Rol solicitado.

        Returns:
            dict: Resultado de autenticación con nivel y mensaje.

        Raises:
            ValueError: Si el nombre de usuario es inválido.
            Exception: Si ocurre un error inesperado en el repositorio.
        """
        if not isinstance(username, str) or not username.strip():
            raise ValueError("El nombre de usuario no puede estar vacío.")

        try:
            authenticated = self.repo.authenticate(username, password)
        except Exception as e:
            raise Exception(f"Error al acceder al repositorio: {e}")

        if not authenticated:
            return {'ok': False, 'lvl': -1, 'msg': 'Auth failed'}

        strategy = get_role_strategy(role)
        return strategy.build_response()
```

No ajuste nada porque todo esta perfecto y bien descriptivo.

Ejercicio 3.2 El problema de seguridad oculto

¿Detectó la IA el problema de seguridad más grave de este código? ¿Cuál es? ¿Cómo lo corrigió o cómo lo corregirías tú? (Pista: tiene que ver con el algoritmo de hash usado.)

El problema de seguridad más grave del código original es el uso de **MD5 para hashear contraseñas**, y sí: la IA lo detectó, lo explicó y lo corrigió en el refactor. Este punto es tan crítico que en seguridad profesional se considera una *vulnerabilidad severa porque es reversible con rainbow tables y vulnerable a fuerza bruta*.

Era muy vulnerable a que un atacante pudiera recuperar la contraseña muy rápido aparte siempre producía el mismo hash.

La IA me ayudó a integrar el bcrypt para mayor seguridad y usamos verify() en lugar de comparar hashes

El nuevo hasher quedó así:

```
Python ^ Copiar

import bcrypt

class Hasher:
    def hash(self, password: str) -> str:
        if not isinstance(password, str) or not password.strip():
            raise ValueError("La contraseña no puede estar vacía.")
        hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
        return hashed.decode()

    def verify(self, password: str, hashed_password: str) -> bool:
        return bcrypt.checkpw(password.encode(), hashed_password.encode())
```

Y el repositorio se actualizó para usar `verify()` en lugar de comparar hashes:

```
Python ^ Copiar

if user == username and self.hasher.verify(password, stored_hash):
    return True
```

Yo cambiaría a argon2 porque es un algoritmo recomendado por OWASP, con migraciones progresiva y costes ajustado.

⚠ Para reflexionar

MD5 no es seguro para contraseñas: es reversible con rainbow tables. La alternativa correcta es bcrypt, argon2 o PBKDF2.



BLOQUE
E 4

Buenas prácticas — Métricas reales

Refactorizar sin medir no tiene sentido. En este bloque vas a comprobar con datos reales si tu refactorización mejoró el código.

🔗 Ejercicio práctico — Abre VS Code para completar esta tarea

Ejercicio 4.1 Mide la complejidad ciclomática

Instala la extensión 'CodeMetrics' en VS Code o usa una herramienta de análisis estático (radon para Python, ESLint para JS). Mide la complejidad ciclomática del código original y del refactorizado:

Función	Complejidad original	Complejidad tras refactorizar	¿Mejoró?
process() / p()	5	2	Si, muchisimo.
UserRepository.authenticate()	N/A	2	Sí
AuthService.process()	N/A	2	Sí
Hasher.hash()	N/A	1	Sí
Hasher.verify()	N/A	1	Sí
get_role_strategy()	N/A	1	Sí
Estrategias de rol	N/A	1	Sí
UserRepository.authenticate()	N/A	2	Sí
AuthService.process()	N/A	2	Sí

¿Qué complejidad ciclomática considera aceptable Clean Code para una función? ¿La IA lo cumplió sin que se lo indicaras?

Entre el 6 y el 10.

Pues no se lo pedi directamente pero si indirectamente en los prompt dando los parametros a seguir. Y si cumplio con todas porque redujo la complejidad del codigo de 5 a 2.

Ejercicio 4.2 Reflexión sobre aceptar cambios sin entender

El módulo dice que aceptar cambios de la IA sin entenderlos es uno de los errores más graves. Durante este ejercicio, ¿hubo algún cambio que la IA propuso y que aceptaste sin entender del todo? ¿Qué pasó cuando lo ejecutaste? ¿Qué harías diferente?

No Siempre tuve el control sobre mi codigo.



BLOQU
E 5

Síntesis — Refactorización guiada por objetivos

En el ejercicio básico el prompt era 'Arregla este código.' Ahora vas a demostrar que sabes escribir prompts de refactorización completos y ejecutarlos en VS Code.

Ejercicio práctico — Abre VS Code para completar esta tarea

Ejercicio 5.1 Tu propio reto de refactorización

Descripción del reto

Busca en tu ordenador (o pide a la IA que genere) un fragmento de código de 20-30 líneas que tenga al menos 3 code smells distintos. Puede ser Python, JavaScript o Java. Aplica el workflow completo de 5 pasos y documenta todo el proceso aquí.

Describe el código que elegiste y los 3 code smells que identificaste antes de pedir ayuda a la IA:

1-Generame un fragmento de codigo legacy de 20 o 30 lineas que tengas 5 code smell distinto en python.

2-Aplicame el workflow 5 pasos a este archivo y documenta todo con docstring.

🌿 Código legacy (28 líneas, 5 code smells reales)

Python ^

📄 Copiar

```
config = {"retry": 3, "timeout": 5}

def send_request(url, params=None):
    if params == None:
        params = {}
    response = None
    for i in range(config["retry"]):
        print("Attempt:", i + 1)
        try:
            result = fake_http_call(url, params)
            if result["status"] == 200:
                response = result
                break
            else:
                print("Bad status:", result["status"])
        except:
            print("Request failed")
    if response == None:
        return {"error": True, "msg": "No response"}
    if "data" in response:
        data = response["data"]
        if type(data) == list:
            for d in data:
                print("Item:", d)
        else:
            print("Single item:", data)
    return response

def fake_http_call(url, params):
    return {"status": 200, "data": ["a", "b", "c"]}
```

Claramente tenemos variables difícil de testear, muchas responsabilidades juntas y la nomenclatura no es clara y legible

Aplicando los 5 pasos.

Paso 1 — Análisis inicial

Code smells detectados (5)

1. **Uso de variable global** (`config`) → acoplamiento fuerte.
2. **Excepción genérica** (`except:`) → oculta errores reales.
3. **Responsabilidades mezcladas** → logging, reintentos, parsing, retorno.
4. **Mutación de parámetros** (`params=None` → `params={}`) → bug clásico.
5. **Condicionales anidados y repetitivos** → difícil de extender y testear.

Objetivo del refactor

- Eliminar globales.
- Separar responsabilidades.
- Manejar errores correctamente.
- Crear funciones pequeñas y testeables.
- Añadir docstrings profesionales.
- Mantener el comportamiento original.

Paso 2 — Plan de refactorización

Acciones planificadas

- Crear una clase `RequestConfig` para encapsular configuración.
- Crear una clase `RequestSender` para manejar reintentos y errores.
- Extraer función `process_response`.
- Sustituir `except:` por excepciones específicas.
- Eliminar prints del core y permitir inyección de logger.
- Evitar mutación de parámetros.
- Añadir docstrings a todas las funciones y clases.

Paso 3 — Ejecución del refactor (código final con docstrings)

Python ^

 Copiar

```
class RequestConfig:
    """
    Configuración para el envío de peticiones HTTP simuladas.

    Atributos:
        retry (int): Número de reintentos permitidos.
        timeout (int): Tiempo máximo de espera simulado.
    """
    def __init__(self, retry: int = 3, timeout: int = 5):
        self.retry = retry
        self.timeout = timeout

class RequestSender:
    """
    Servicio responsable de enviar peticiones HTTP con reintentos,
    manejo de errores y procesamiento de respuestas.

    Este componente encapsula la lógica que antes estaba mezclada
    dentro de una única función legacy.
    """
    def __init__(self, config: RequestConfig, logger=print):
        """
        Inicializa el servicio con configuración y logger inyectable.

        Args:
            config (RequestConfig): Configuración de reintentos y timeout.
            logger (callable): Función para registrar mensajes (por defecto print).
        """
        self.config = config
        self.logger = logger

    def send(self, url: str, params: dict | None = None) -> dict:
        """
        Envía una petición simulada con reintentos y manejo de errores.

        Args:
            url (str): URL destino.
            params (dict | None): Parámetros de la petición.

        Returns:
            dict: Respuesta procesada o error estructurado.
        """
        params = params or {}
        response = None

        for attempt in range(self.config.retry):
            self.logger(f"Attempt: {attempt + 1}")
            try:
                result = fake_http_call(url, params)
                if result.get("status") == 200:
                    response = result
                    break
                self.logger(f"Bad status: {result.get('status')}")
            except Exception as e:
                self.logger(f"Request failed: {e}")

        if response is None:
            return {"error": True, "msg": "No response"}
```

```

        return self.process_response(response)

def process_response(self, response: dict) -> dict:
    """
    Procesa la respuesta HTTP simulada.

    Args:
        response (dict): Respuesta cruda.

    Returns:
        dict: Respuesta final procesada.
    """
    data = response.get("data")

    if isinstance(data, list):
        for item in data:
            self.logger(f"Item: {item}")
    else:
        self.logger(f"Single item: {data}")

    return response

def fake_http_call(url, params):
    """
    Simula una llamada HTTP devolviendo datos estáticos.

    Args:
        url (str): URL destino.
        params (dict): Parámetros enviados.

    Returns:
        dict: Respuesta simulada.
    """
    return {"status": 200, "data": ["a", "b", "c"]}

```

Paso 4 — Validación

¿Se mantiene el comportamiento original?

- ✓ Reintentos → Sí
- ✓ Logs de intentos → Sí
- ✓ Logs de errores → Sí
- ✓ Procesamiento de listas y elementos → Sí
- ✓ Retorno final → Igual
- ✓ Sin prints en el core → Ahora se usa logger inyectable
- ✓ Sin globales → Config encapsulada
- ✓ Sin mutación de parámetros → Corregido
- ✓ Sin `except:` genérico → Corregido

Tests conceptuales

- `send()` devuelve error si no hay respuesta.
- `send()` procesa listas igual que antes.
- `process_response()` imprime cada item.
- `fake_http_call()` sigue funcionando igual.

Todo consistente.

Paso 5 — Documentación final

Resumen del refactor

- Se eliminaron 5 code smells críticos.
- Se aplicó separación de responsabilidades.
- Se introdujo una arquitectura orientada a objetos.
- Se eliminó el uso de variables globales.
- Se mejoró la testabilidad mediante logger inyectable.
- Se añadieron docstrings profesionales.
- Se mantuvo el comportamiento original.

Beneficios obtenidos

- Código más mantenible.
- Código más extensible.
- Código más seguro.
- Código más testeable.
- Código más legible.

Paso del workflow	Tipo de refactorización aplicado	Resultado obtenido
Análisis inicial	Identificación de <i>code smells</i> : variable global, excepción genérica, responsabilidades mezcladas, condicionales anidados, mutación de parámetros	Se detectaron 5 problemas estructurales que impedían testabilidad, mantenibilidad y extensibilidad
Plan	Diseño de una arquitectura modular: extracción de funciones, encapsulación de configuración, introducción de clases, logger inyectable, manejo seguro de errores	Se definió un plan claro para separar responsabilidades y eliminar los <i>code smells</i> sin alterar el comportamiento
Ejecución	Refactorización completa: creación de RequestConfig, RequestSender, process_response, eliminación de globales, sustitución de except: por excepciones controladas, docstrings profesionales	El código quedó limpio, modular, extensible, testeable y sin duplicación ni acoplamiento innecesario
Validación	Pruebas conceptuales del flujo: reintentos, logs, procesamiento de datos, retorno de errores, comportamiento idéntico al original	Se confirmó que el comportamiento observable se mantiene, pero con una arquitectura mucho más robusta
Documentación	Docstrings detallados en todas las clases y métodos, explicación del diseño, responsabilidades claras y comentarios de reglas de negocio	El código quedó documentado profesionalmente, facilitando mantenimiento y futuras extensiones

¿Qué tipo de refactorización fue más difícil de conseguir con la IA? ¿Por qué? ¿Qué estrategia de prompt funcionó mejor?

La refactorización más difícil de conseguir con la IA fue la **separación de responsabilidades mediante introducción de nuevas abstracciones** (clases, funciones auxiliares, eliminación de

globales y reorganización del flujo). Este tipo de refactor requiere que la IA **entienda el propósito del código**, no solo lo reescriba, y eso siempre es más complejo que aplicar cambios superficiales como renombrar variables o eliminar duplicación.

El mejor prompt es guiarla paso a paso mientras revisas todo lo que te da y uno decide que cambiar y que no.

Checklist de entrega

- ☐ Ejercicio 1.1: tabla de code smells del código p() completada
- ☐ Ejercicio 1.2: tabla de nomenclatura con justificaciones
- ☐ Ejercicio 1.3: simplificación ejecutada en VS Code y reflexión escrita
- ☐ Ejercicio 2.1: funciones modularizadas documentadas en tabla
- ☐ Ejercicio 2.2: patrón Strategy implementado con ventajas justificadas
- ☐ Ejercicio 3.1: workflow de 5 pasos sobre código process() completo
- ☐ Ejercicio 3.2: problema de seguridad identificado y solución propuesta
- ☐ Ejercicio 4.1: métricas de complejidad medidas antes/después
- ☐ Ejercicio 5.1: código propio refactorizado con workflow documentado